

CS360 Introduction to Database

Query Execution

Min-Soo Kim

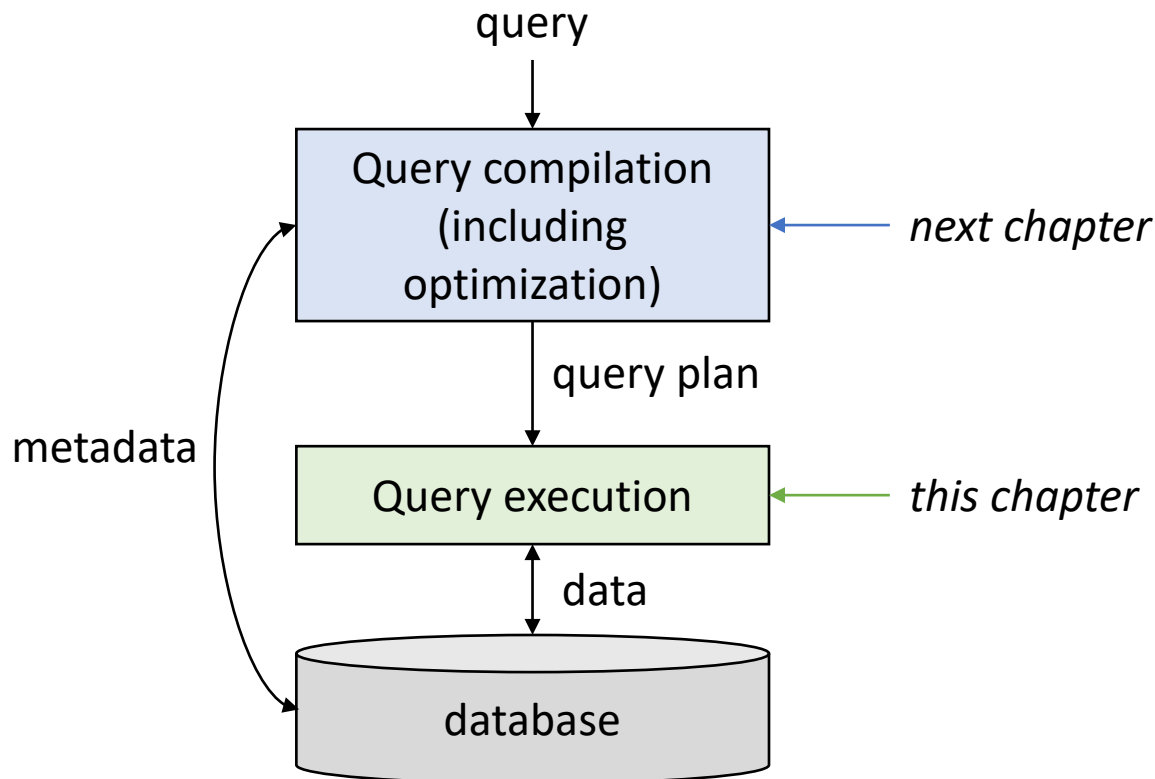
School of Computing, KAIST

Outline

- Query compilation
- Physical query plan
- Estimation of cost
- One-Pass algorithms
- Two-Pass algorithms
- Buffer management

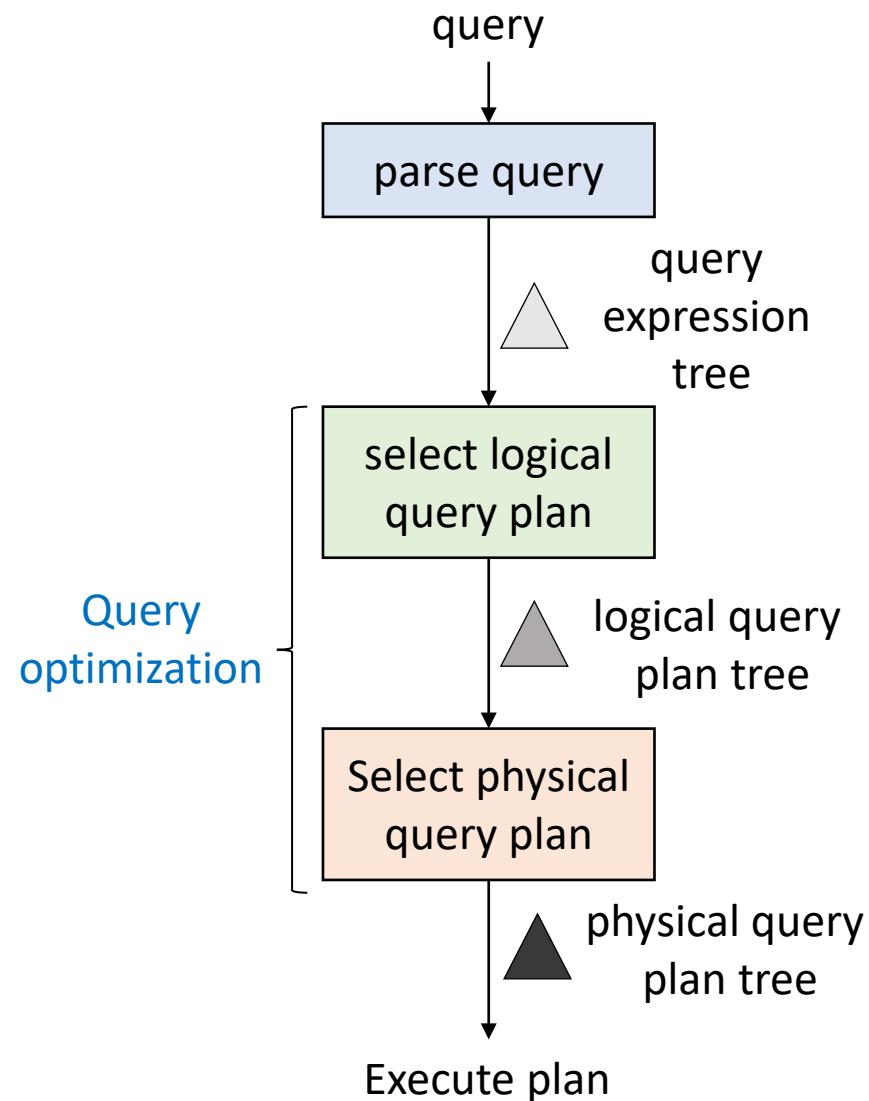
Query processor

- Turn **user queries** (including data-modification commands) into **a sequence of database operations**
- **Execute the operations**



Query compilation

1. **Parsing:** constructs a **parse tree** for the query
2. **Logical plan generation:** converts the parse tree to a **logical query plan**
 - query rewrite: transforms an initial plan into a more efficient equivalent plan
3. **Physical plan generation:** turn the logical plan into a **physical query plan**
 - selecting algorithms and an order

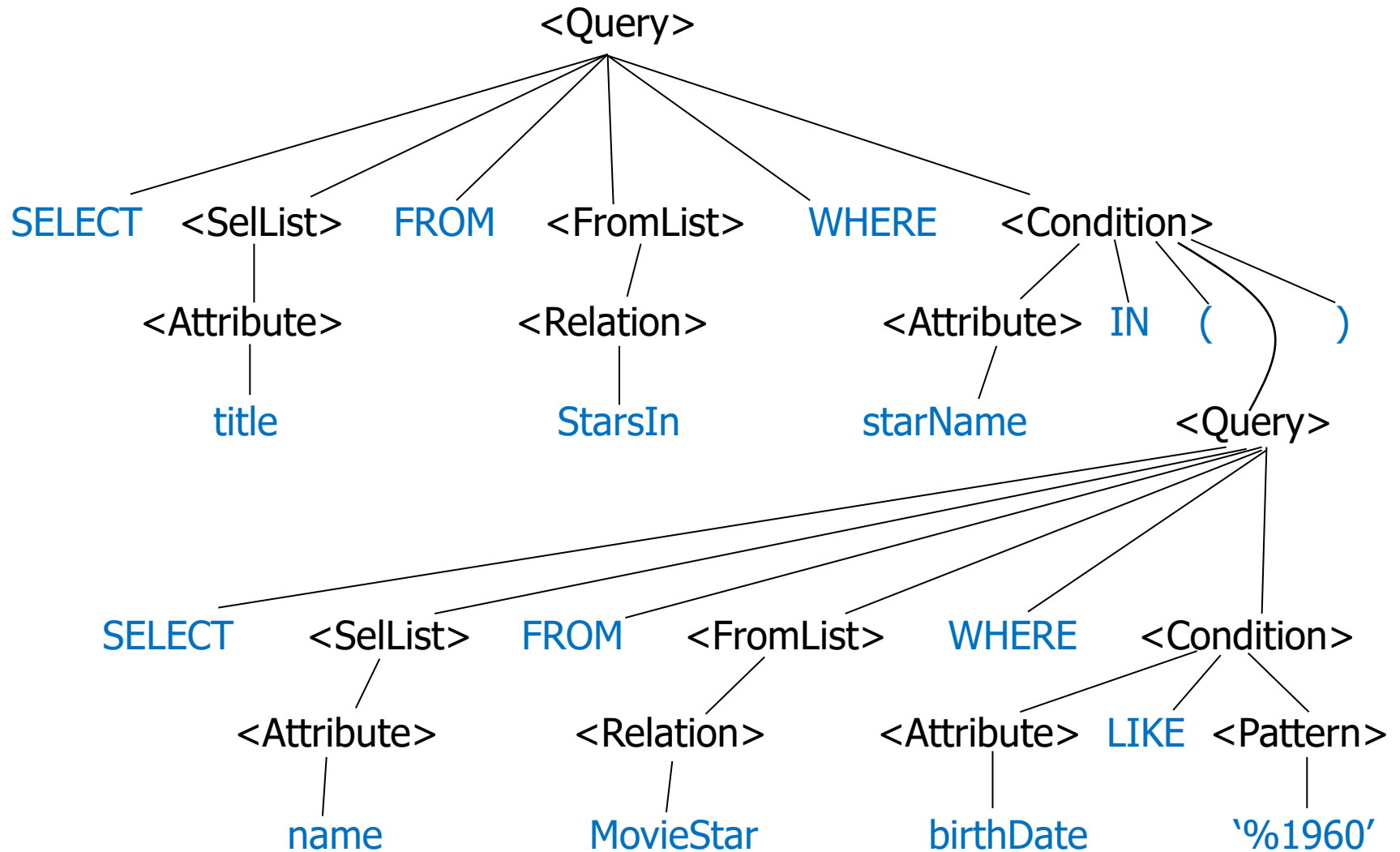


Example: SQL query

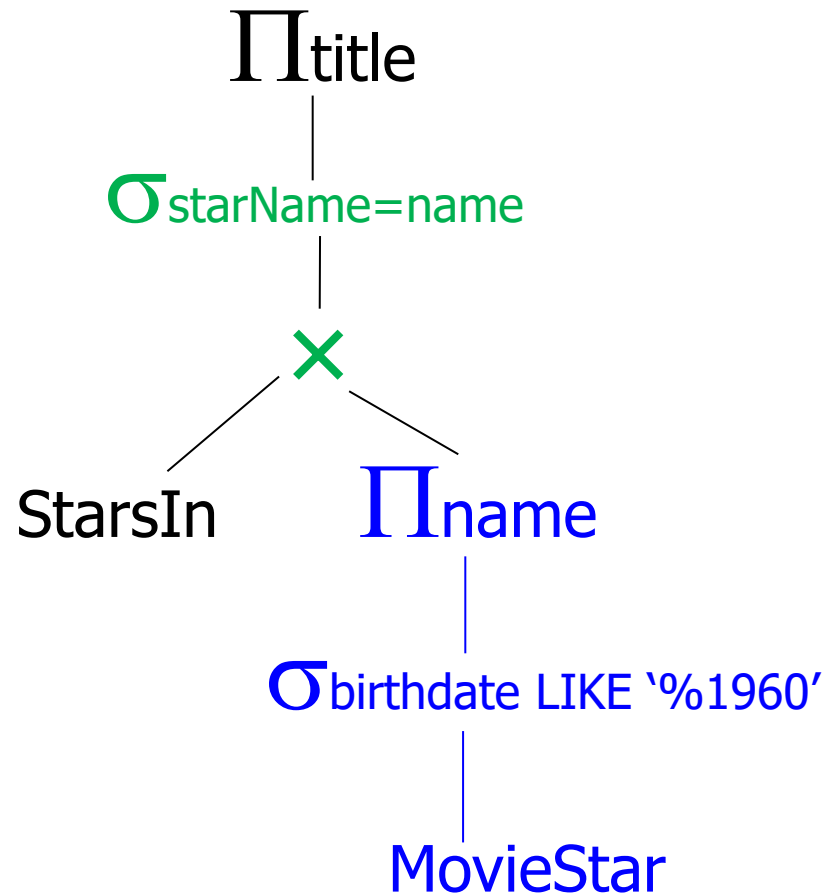
```
SELECT title
FROM StarsIn
WHERE starName IN (
    SELECT name
    FROM MovieStar
    WHERE birthdate LIKE '%1960'
);
```

(Find the movies with stars born in 1960)

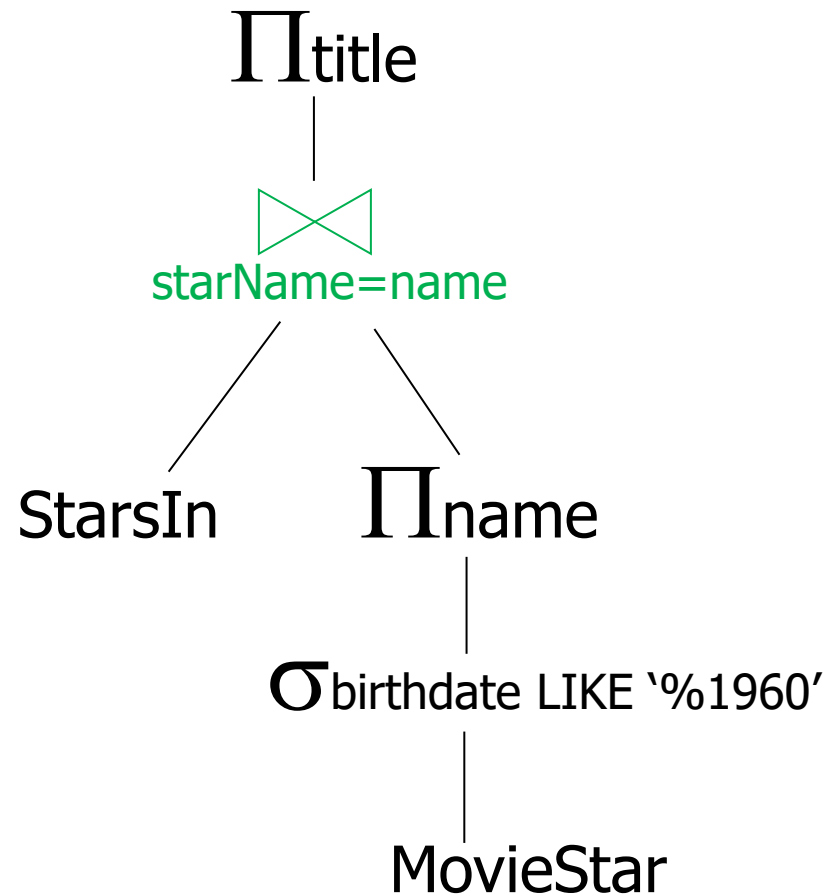
Example: Parse Tree



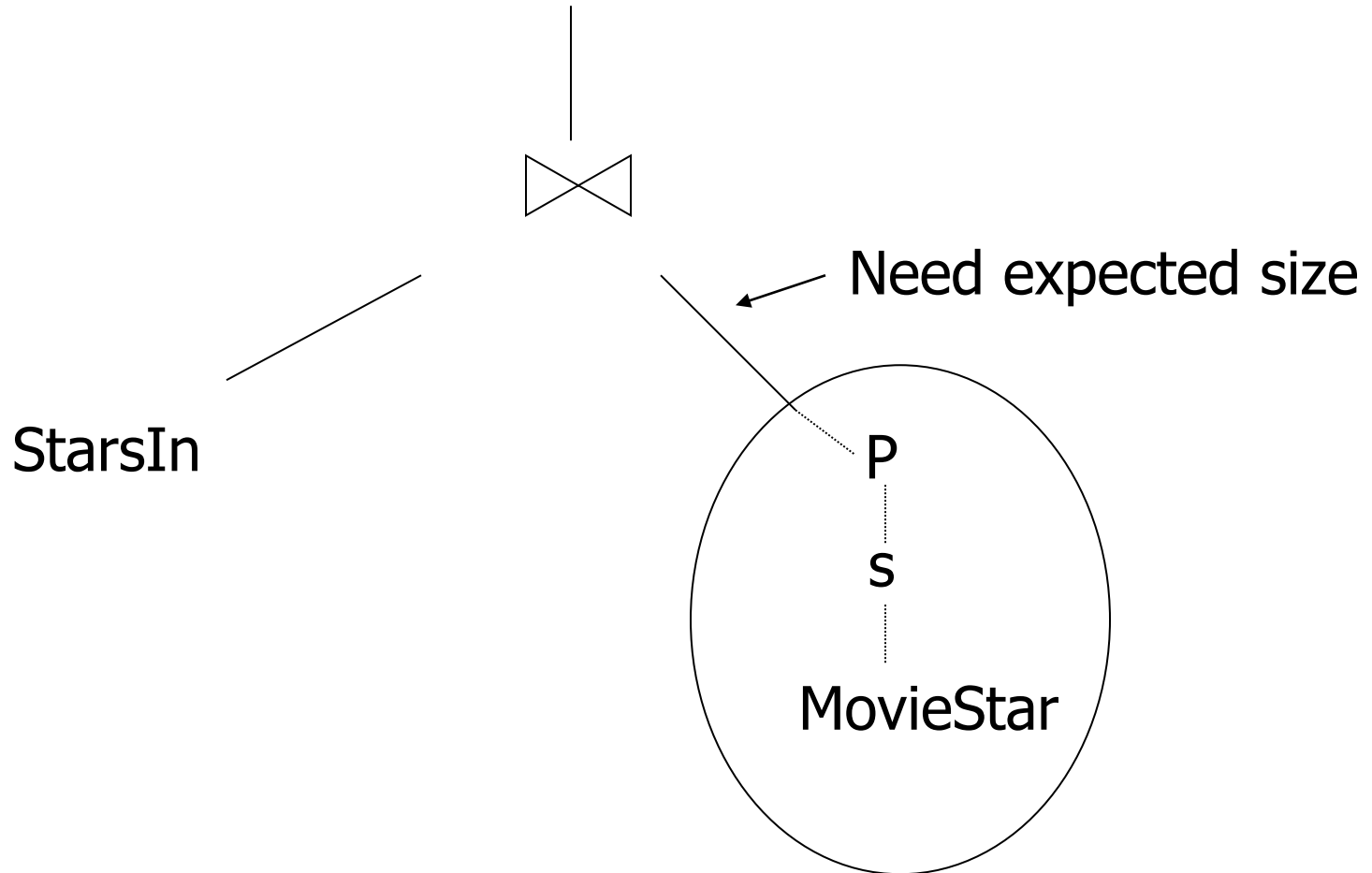
Example: Logical Query Plan



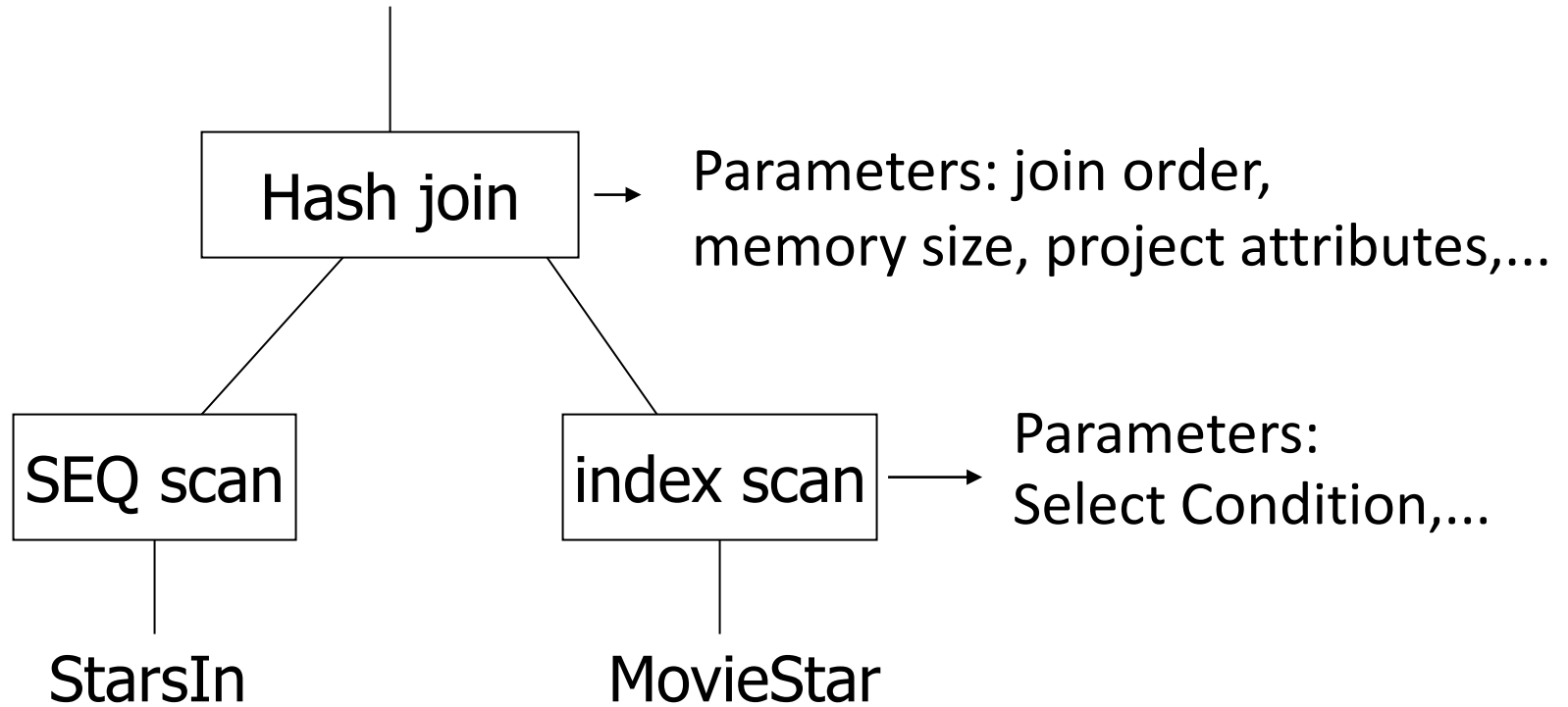
Example: Improved Logical Query Plan



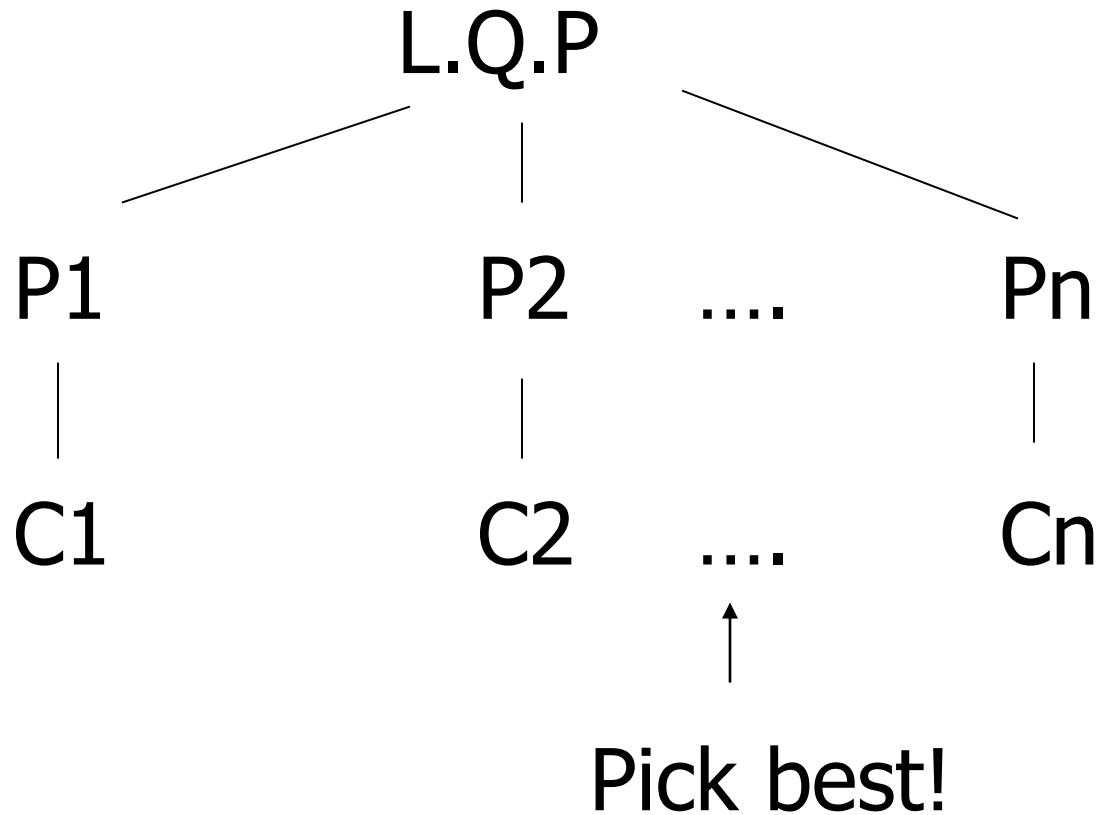
Example: Estimate Result Sizes

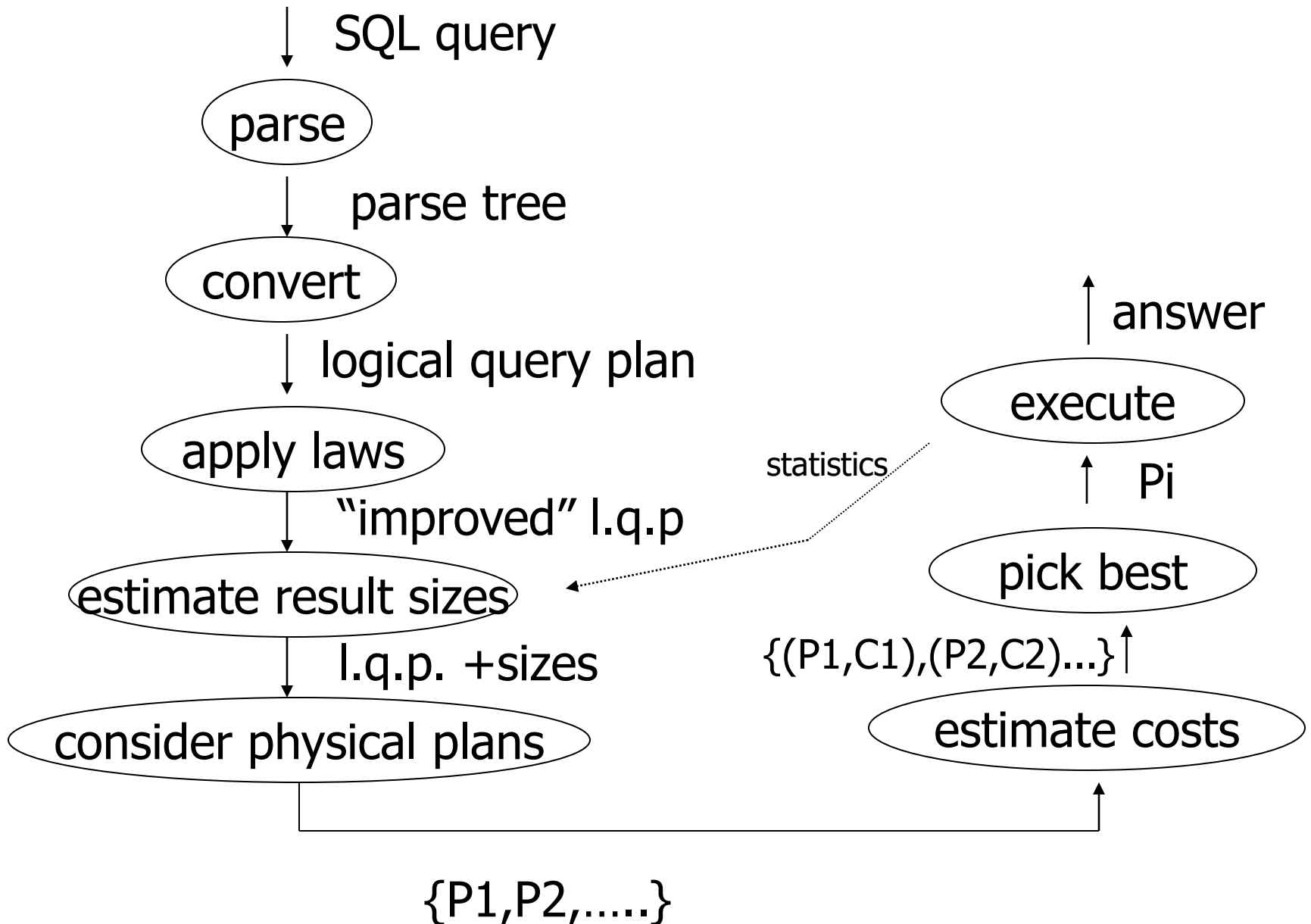


Example: One Physical Plan



Example: Estimate costs





Physical Query Plan and Scan Operator

To select the best query plan

- We need to decide
 - ① which of the algebraically equivalent plans of a query leads to the most efficient algorithm for answering the query?
 - ② what algorithm should we use to implement each operation of the selected plan?
 - ③ how should the operations pass data from one to the other?
 - e.g., in a pipelined fashion in main-memory buffers, or via the disk?
- The choices depends on the metadata
 - e.g., size of each relation, statistics, existence of indexes, layout of data on disk, ...

Operators of physical query plan

- Physical operator

- particular implementations for one of the operations of relational algebra
- we need physical operators for some tasks that do not related to relational algebra
- e.g., hash join, sequential(table) scan, index scan, ...

- Iterator

- we scan a table to bring each tuple of the table into main memory
- method by which the operators of a physical query plan can pass requests for tuples and answers among the operators

Iterators of physical operators

- Group of three operations
- `Open()`
 - starts the process of getting tuples, but does not get a tuple
 - initializes any data structures needed to perform the operation
- `GetNext()`
 - returns either `the next tuple` or *NotFound*
 - adjusts data structures as necessary to allow subsequent tuples to be obtained
- `Close()`
 - ends the iteration after all tuples have been obtained

Iteration vs. materialization (within a query plan)

- Materialization

- the result of each operator is produced in its entirety
- then, stored on disk or takes up space in main memory

- Iteration (pipelining)

- many operations are active at once
- tuples pass among operators, thus reducing the need for storage or main memory
- not all physical operators support the iteration approach
 - performed in the materialization approach

Scanning tables

- Most basic operator: read the entire tuples of a relation R
 - can read only tuples that satisfy the predicates in the query
- Table scan
 - the blocks containing the tuples of R are known to the system
 - read the blocks one by one
- Index scan
 - an index may be on an attribute of R
 - useful for the selection operator on the attribute (particular value or range of values)

Sort scan

- What
 - take a relation R and a specification of the attributes on which the sort is to be made
 - produce R in that sorted order
- When
 - explicitly due to ORDER BY clause
 - implicitly due to some physical operators requiring one or both arguments (i.e., relations) to be sorted
- How
 - index scan using an index (e.g., B-tree) on the attribute
 - table scan with in-memory sort or multiway merge-sort

Estimation of Cost

Cost model

- Choosing good physical-plan operators is essential
 - we must be able to estimate the cost of each operator
- Measure: # of disk I/O's (for disk-based systems)
 - $\text{speed(I/O)} \ll \text{speed(main memory)} \ll \text{speed(processor)}$
- Assumption
 - we do not consider the cost of writing the query answer
 - the result of an operator may be stored in disk or not be stored in disk
 - the cost of storing depends on the size of the result, not on how the result was computed

Estimation of cost

- Estimate the cost based on parameters (statistics)
- Parameters
 - number of blocks, M , in main memory that an operator uses (i.e., buffer)
 - M could be much smaller than the total main memory due to the situation where many operations share the main memory
 - M may be decided during execution
 - size of the arguments (relations) of the operator
 - distribution of data in a relation
 - computed periodically to help the optimizer choose physical operators

Symbols for size and distribution of a relation R

- $B(R)$: number of blocks needed to hold all the tuples of R
 - we usually assume that R is **clustered**, i.e., packed in as small as possible number of blocks
- $T(R)$: number of tuples in R
 - $T(R) / B(R)$: number of tuples that can fit in one block
- $V(R, a)$: number of distinct values in a column **a** in R
 - $V(R, [a_1, a_2, \dots, a_n])$: number of **distinct n-tuples** in the columns of R for the list of attributes $[a_1, a_2, \dots, a_n]$

Cost of scanning

- Table scan
 - $B(R)$ disk I/O's, if R is clustered
 - but, much higher number of disk I/O's, if R is not clustered
 - at most $T(R)$ disk I/O's, if R is distributed among tuples of other relations
- Index scan
 - index on R usually occupies many fewer than $B(R)$ blocks
 - at least $B(R)$ blocks, if the entire R is scanned, and R is clustered
- Sort scan
 - $B(R)$ disk I/O's, if R fits in main memory

Categorization of algorithms

- Choice of algorithm for each logical operator
 - essential part of query optimization
 - done in the process of transforming a logical query plan into a physical query plan
- Categorization 1: iteration-based, sorting-based, hash-based, index-based
- Categorization 2 (by the degree of difficulty and cost)
 - one-pass: reading data only once from disk
 - two-pass: reading data two times from disk
 - multi-pass: recursive generalization of two-pass algorithms

One-Pass Algorithms

	iteration-based	sorting-based	hash-based	index-based
one-pass	selection, projection, duplicate-elimination, grouping, union, intersection, difference, join, product			
two-pass				

Three groups of one-pass methods

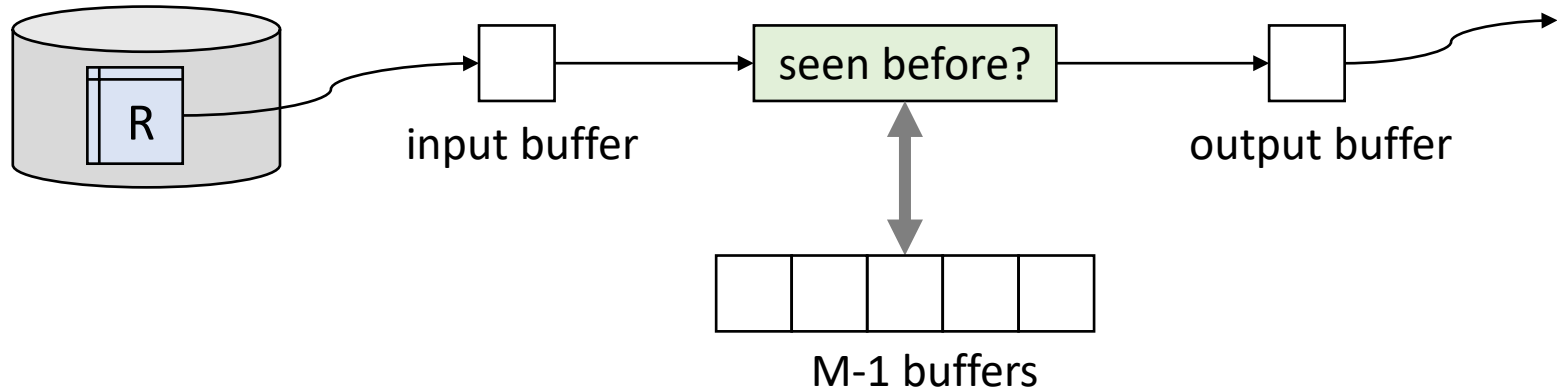
- Tuple-at-a-time (unary operations)
 - operators: selection (σ), projection (π)
 - can read a block at a time using one-block buffer and produce output
- Full-relation (unary operations)
 - operators: duplicate-elimination (δ), grouping (γ)
 - require seeing all or most of the tuples in memory at once
- Full-relation (binary operations)
 - operators: union, intersection, difference, join, product
 - can use one-pass, if at least one argument to be limited to M

Tuple-at-a-time operations: σ , π

- Cost of operators
 - $B(R)$, if R is clustered
 - $T(R)$, if R is not clustered
- Exception: retrieve only a subset of the blocks holding R
 - if selection uses **index scan**, and the condition **compares a constant** to an attribute that has an index
- In practice, we can speed up reading more blocks to buffer
 - if R is stored on consecutive blocks within cylinders

Unary and full-relation operations: δ

- Duplicate elimination is done using data structures in main memory (e.g., hash table)
 - neglect some space overhead for data structures for simplicity
- Assumption: $B(\delta(R)) \leq M$
 - M-1 blocks for holding a single copy of every tuple seen so far
 - thrashing will happen if $B(\delta(R)) > M$



Unary and full-relation operations: γ

```
SELECT S.rating, MIN (S.age)
FROM Sailors S
WHERE S.age >= 18
GROUP BY S.rating
HAVING COUNT (*) > 1
```

- **Grouping operation γ**
 - zero or more grouping attributes
 - presumably, one or more aggregated attributes
- **Group entry:** <grouping attributes' value, aggregated value>
 - e.g., <S.rating, MIN(S.age)> : <7, 35.0>, <10, 16.0>, ...
 - **MIN(x) / MAX(x)**: change the minimum / maximum of the group
 - **COUNT**: add one for each tuple of the group
 - **SUM(x)**: add the value of attribute a to the sum for the group
 - **AVG(x)**: maintain two accumulations, count and sum
- Group entries are maintained in **hash tables ($\leq M$)**
 - search key: the grouping attributes only

Binary and full-relation operations

- Must distinguish set- and bag-versions
 - e.g., $-_S$ for set difference, $\cup_B$ for bag union
- Bag union: $R \cup_B S$
 - # of disk I/O's : $B(R) + B(S)$
- Set union / intersection / difference: $\min(B(R), B(S)) \leq M$
 - ① read a smaller operand into M-1 buffers (assume $B(S) < B(R)$)
 - ② build a hash table on S
 - ③ read a larger operand, R, using 1-block buffer
 - compute $R -_S S$ and $S -_S R$ differently ($-_S$ is not commutative)

- **Bag intersection:** $\min (B(R), B(S)) \leq M$
 - build a hash table on S with a *count* for each distinct tuple
 - output t of R and decrement its count by 1, if t appears in S, and its count is positive
 - ignore t of R, if t does not occur in S, or if its count is zero
- **Bag difference ($S -_B R$):** $\min (B(R), B(S)) \leq M$
 - build a hash table on S with a *count* for each distinct tuple
 - decrement R.t's count, if t appears in S
 - output each S.t whose count is positive, the count times
- **Bag difference ($R -_B S$):** $\min (B(R), B(S)) \leq M$
 - build a hash table on S with a *count* for each distinct tuple
 - output t of R, if t does not occur in S, or if its count is zero
 - ignore t of R and decrement its count by 1, if t appears in S

- **Product:** $\min (B(R), B(S)) \leq M$
 - read S into M-1 buffer (no special data structure)
 - read t of R, concatenate t with each tuple of S, and output it
- **Natural join** $(R(X,Y) \bowtie S(Y,Z))$: $\min (B(R), B(S)) \leq M$
 - build a hash table on S with using Y as the search key
 - output (X,Y,Z), if R.t exists in the hash table s.t. $R.t[Y] = S.t[Y]$
- **Equijoin**
 - can be done in the same way with natural join after renaming
- **Theta-join**
 - can be replaced by equijoin (or product) followed by a selection
 - selection: the conditions that cannot be expressed in equijoin

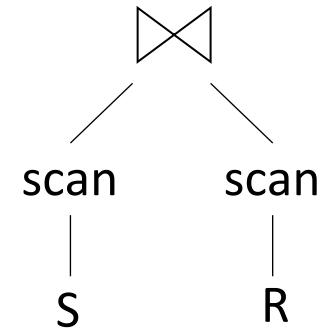
Two-Pass Algorithms

	iteration-based	sorting-based	hash-based	index-based
one-pass	selection, projection, duplicate-elimination, grouping, union, intersection, difference, join, product			
two-pass	nested-loop joins			

Nested-loop joins

- One-and-a-half pass
 - one-pass for one argument (relation)
 - but, multi-pass for the other argument
- Pros: can be used for relations of any size
 - S does not necessarily fit in main memory
- Cons: slow
- We consider $R(X,Y) \bowtie S(Y,Z)$ in next slides

Tuple-based nested-loop join



- # of I/O's: $T(R)T(S)$
- Fit well into the iterator framework
 - avoid storing intermediate relations on disk in some situations

```
getNext() { /* S is outer, R is inner */
    repeat {
        r = R.getNext();
        if (r = EOR) { /* End of Relation R */
            R.Close();
            s = S.getNext();
            if (s = EOR) return EOR; /* End of R and S */
            R.Open();
            r = R.getNext();
        }
    } until (r.Y = s.Y);
    return <r,s>;
}
```

Block-based nested-loop join

- Improve performance by
 - organizing access to both relations **by blocks**
 - using **M-1 buffers** to store tuples of S, (i.e., **outer relation**)
 - build a hash table for M-1 buffers
 - using **one block** to read tuples of R (i.e., **inner relation**)
- Example: $B(R)=1000$, $B(S)=500$, $M=101$
 - # of I/O for reading S: $100 \times 5 = 500$
 - # of I/O for reading R: $1000 \times 5 = 5000$
 - total # of I/O: **5500**

Reverse join order

- $M-1$ buffers to store tuples of R , (i.e., outer relation)
- One buffer to read tuples of S (i.e., inner relation)
- Example: $B(R)=1000$, $B(S)=500$, $M=101$
 - # of I/O for reading R : $100 \times 10 = 1000$
 - # of I/O for reading S : $500 \times 10 = 5000$
 - total # of I/O: 6000
- Using a smaller relation in outer loop is better
- Cost of nested-loop join is approximately $B(R)B(S)/M$

Two-Pass Algorithms

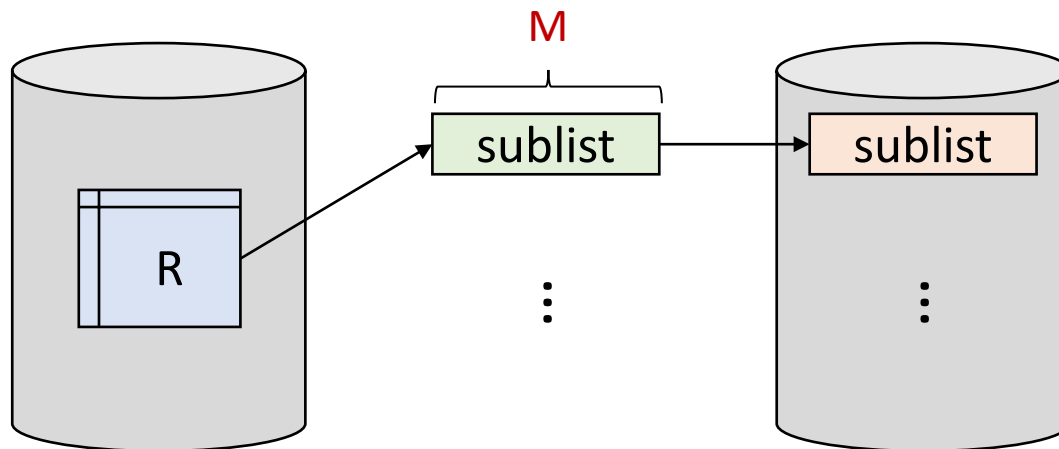
	iteration-based	sort-based	hash-based	index-based
one-pass	selection, projection, duplicate-elimination(δ), grouping(γ), union, intersection, difference, join, product			
two-pass	nested-loop joins	$\tau, \gamma, \delta,$ $\cup, \cap, -,$ sort-based join		

Two-pass algorithms

- Principle
 - ① data from the operand relations is **read into** memory buffer, processed in some way, and **written out** to disk again
 - ② written data is **re-read from disk to complete the operation**
- Why two-pass algorithms?
 - **two passes are usually enough even for very large relations**
 - generalizing to more than two passes is not hard
- Sorting operator $\tau(R)$, $B(R) > M$

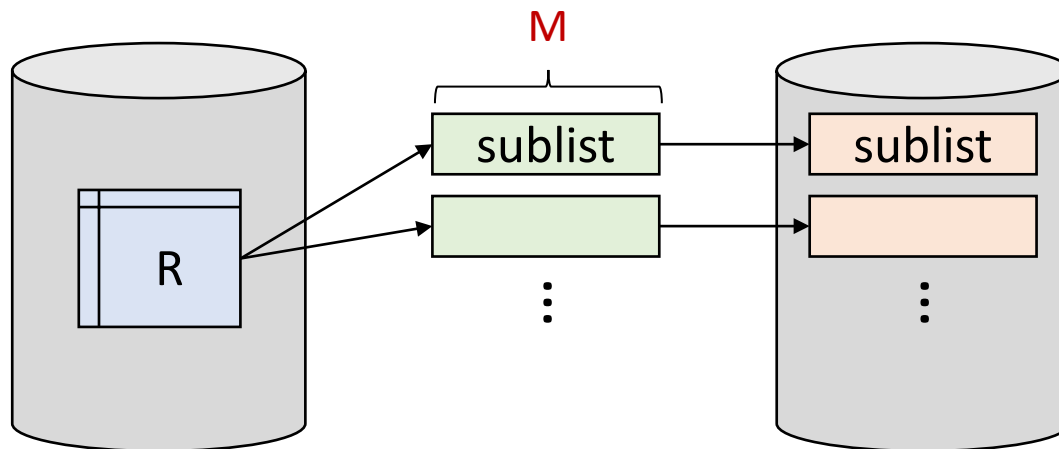
Two-phase, multiway merge-sort (2PMMS)

- Phase 1
 - repeatedly fill **M buffers** with new tuples from R and sort them using in-memory sorting algorithm
 - write out each **sorted sublist** to disk
- Phase 2
 - merge the sorted sublists (at most **M-1 sorted sublists**)
 - one block for output



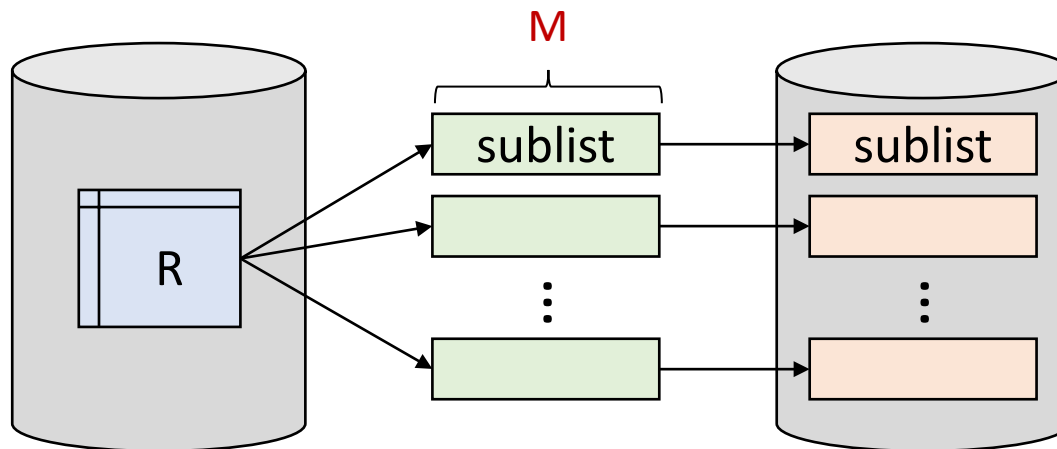
Two-phase, multiway merge-sort (2PMMS)

- Phase 1
 - repeatedly fill **M buffers** with new tuples from R and sort them using in-memory sorting
 - write out each sorted sublist to disk
- Phase 2
 - merge the sorted sublists (at most **M-1 sorted sublists**)
 - one block for output



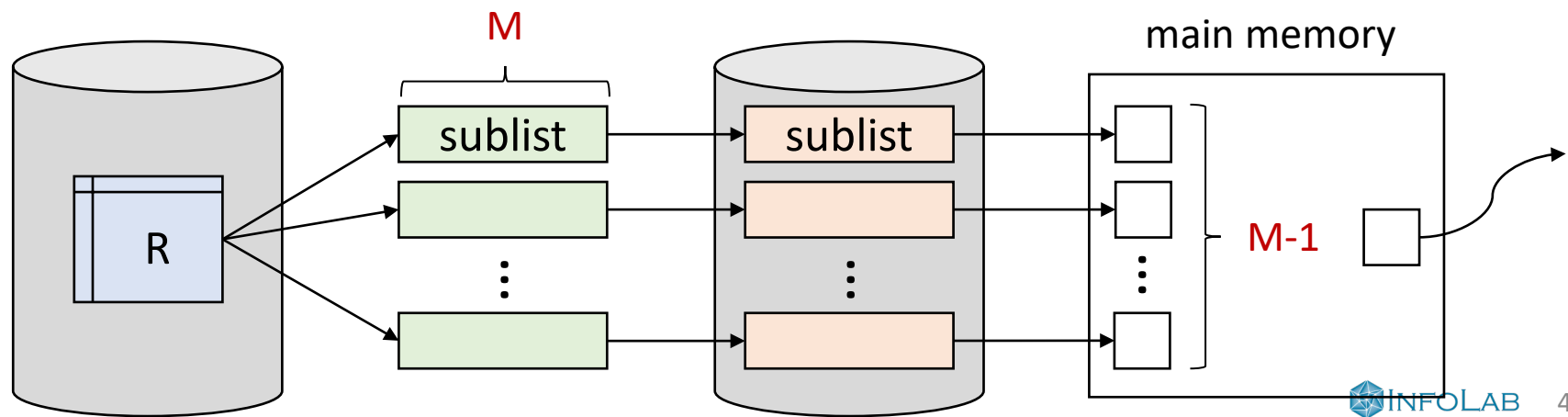
Two-phase, multiway merge-sort (2PMMS)

- Phase 1
 - repeatedly fill **M buffers** with new tuples from R and sort them using in-memory sorting
 - write out each sorted sublist to disk
- Phase 2
 - merge the sorted sublists (at most **M-1 sorted sublists**)
 - one block for output



Two-phase, multiway merge-sort (2PMMS)

- Phase 1
 - repeatedly fill **M buffers** with new tuples from R and sort them using in-memory sorting
 - write out each sorted sublist to disk
- Phase 2
 - merge the sorted sublists (at most **M-1 sorted sublists**)
 - one block for output



Space and time cost of 2PMMS

- # of sublists : $B(R)/M$
- $B(R)/M \leq M-1$ (due to phase 2)
- $B(R) \leq M(M-1)$, i.e., $B \leq M^2$
- e.g., size of block = 64KB, total memory = 1GB, $M = 16K$, then $(16K)^2 = 2^{28}$, i.e., $64KB * 2^{28} = 4TB$ can be sorted
- Larger R can be sorted by using 2PMMS recursively
- Total 3B disk I/O's (2B for read and write, 1B for read)

Sort-based duplicate elimination (δ)

- Pass 1
 - make sorted sublists of R using 1st phase of 2PMMS
- Pass 2
 - read one block from each sorted sublist (as in 2PMMS)
 - output only one copy of a tuple t and eliminate all other occurrences of the tuple t

	Total disk I/O's	Max size of R that we can eliminate duplicates
One-pass	$B(R)$	$B(R) \leq M$
Sort-based two-pass	$3B(R)$	$B(R) \leq M^2$

Sort-based grouping and aggregation

- Pass 1
 - make sorted sublists of R using grouping attributes as sort key
- Pass 2
 - read one block from each sorted sublist (as in 2PMMS)
 - compute aggregation for the tuples having the same key

	Total disk I/O's	Max size of R that we can group and aggregate
One-pass	$B(R)$	$B(R) \leq M$
Sort-based two-pass	$3B(R)$	$B(R) \leq M^2$

Sort-based union, intersection, and difference

- Pass 1
 - make sorted sublists from both R and S
- Pass 2
 - read one block from each sorted sublist of R and S
 - $\cup_S$: output one copy of a tuple t and remove all other copies of t
 - $\cap_S$: output a tuple t , if it appears in both R and S
 - $-_S$: output a tuple t , if and only if it appears in R but not in S

	Total disk I/O's	Max sizes of R and S that we can do union, intersection, and difference
One-pass	$B(R) + B(S)$	$\min(B(R), B(S)) \leq M$
Sort-based two-pass	$3(B(R) + B(S))$	$B(R) + B(S) \leq M^2$

Basic sort-based join algorithms $R(X,Y) \bowtie S(Y,Z)$

- Pass 1
 - ① sort R and S **completely** by using Y as sort key (i.e., $3B(R)+3B(S)$)
 - ② **write the result to disk** (i.e., $B(R)+B(S)$)
- Pass 2
 - merge the sorted R and S (using only two buffers) (i.e., $B(R)+B(S)$)
- Example: $B(R)=1000$, $B(S)=500$, $M=101$
 - # of I/O for sorting R: $4 \times 1000 = 4000$
 - # of I/O for sorting S: $4 \times 500 = 2000$
 - # of I/O for merge: $1000 + 500 = 1500$
 - total # of I/O in **(basic) sort-based join**: **7500**
 - c.f., total # of I/O in **block-based nested-loop join**: **5500**

Nested-loop join vs. sort-based join

- Nested-loop join is inherently quadratic: $O(B(R)B(S))$
- Sort-based join is linear: $O(B(R) + B(S))$
- Example: $B(R)=10000$, $B(S)=5000$, $M=101$
 - # of I/O for reading S: $100 \times 50 = 5,000$
 - # of I/O for reading R: $10000 \times 50 = 500,000$
 - total # of I/O in nested-loop join: 505,000

nested-loop join

 - # of I/O for sorting R: $4 \times 10000 = 40,000$
 - # of I/O for sorting S: $4 \times 5000 = 20,000$
 - # of I/O for merge: $10000 + 5000 = 15,000$
 - total # of I/O in sort-based join: 75,000

sort-based join

More efficient sort-based join (**sort-merge-join**)

- Pass 1
 - make sorted sublists for both R and S
- Pass 2
 - read one block from each sorted sublist of R and S
 - merge the sublists of R and S
- Example: $B(R)=1000$, $B(S)=500$, $M=101$
 - # of I/O for creating the sublists of R: $2 \times 1000 = 2000$
 - # of I/O for creating the sublists of S: $2 \times 500 = 1000$
 - # of I/O for merge: $1000 + 500 = 1500$
 - total # of I/O in **sort-merge-join**: **4500** (< 7500 in basic method)

Summary of sort-based algorithms

Operators	Total disk I/O's	Max sizes of R (and S) that we can perform the operation
τ, γ, δ	$3B(R)$	$B(R) \leq M^2$
$\cup, \cap, -$	$3(B(R) + B(S))$	$B(R) + B(S) \leq M^2$
 (basic)	$5(B(R) + B(S))$	$\max(B(R), B(S)) \leq M^2$
 (sort-merge -join)	$3(B(R) + B(S))$	$B(R) + B(S) \leq M^2$

Two-Pass Algorithms

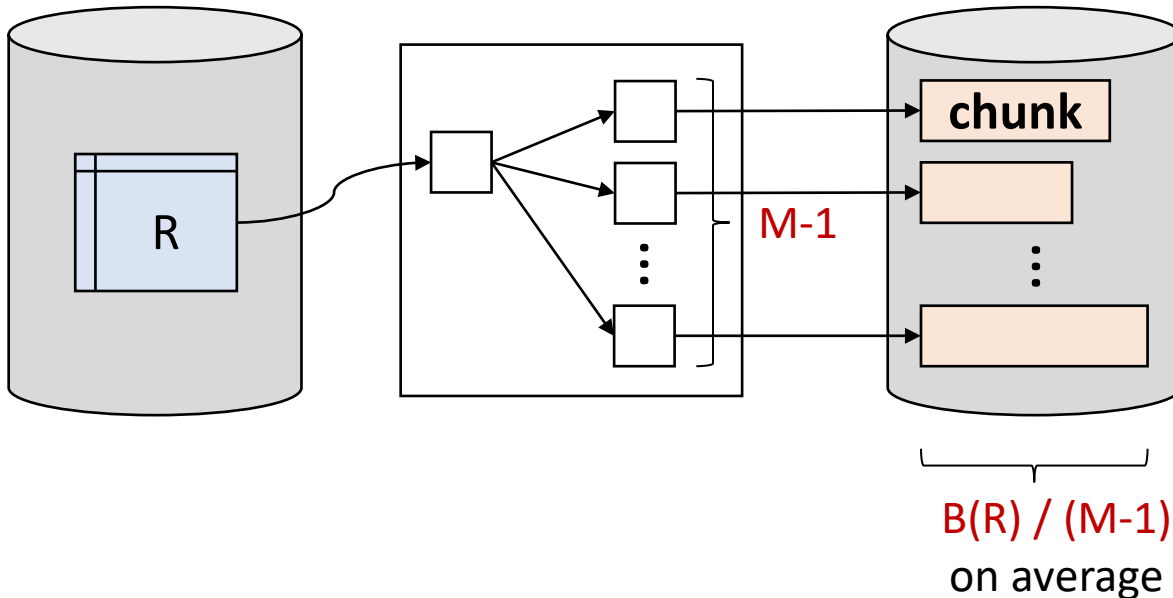
	iteration-based	sort-based	hash-based	index-based
one-pass	selection, projection, duplicate-elimination, grouping, union, intersection, difference, join, product			
two-pass	nested-loop joins	$\tau, \gamma, \delta,$ $\cup, \cap, -$, sort-based join	$\gamma, \delta,$ $\cup, \cap, -$, hash-based join	

Basic idea

- Pass 1: **hash all the tuples** of an argument relation(s)
 - using an appropriate hash key (sort key in sort-based methods)
 - all the tuples that need to be considered together **fall into the same bucket**
- Pass 2: **perform the operation** on a bucket (or **a pair of buckets**) at a time
- # of buckets: M
 - **partition** the relation R into $M-1$ buckets, i.e., $R_0, R_1, \dots, R_{M-2}$
 - $h(t)$: hash function, $t \in R$

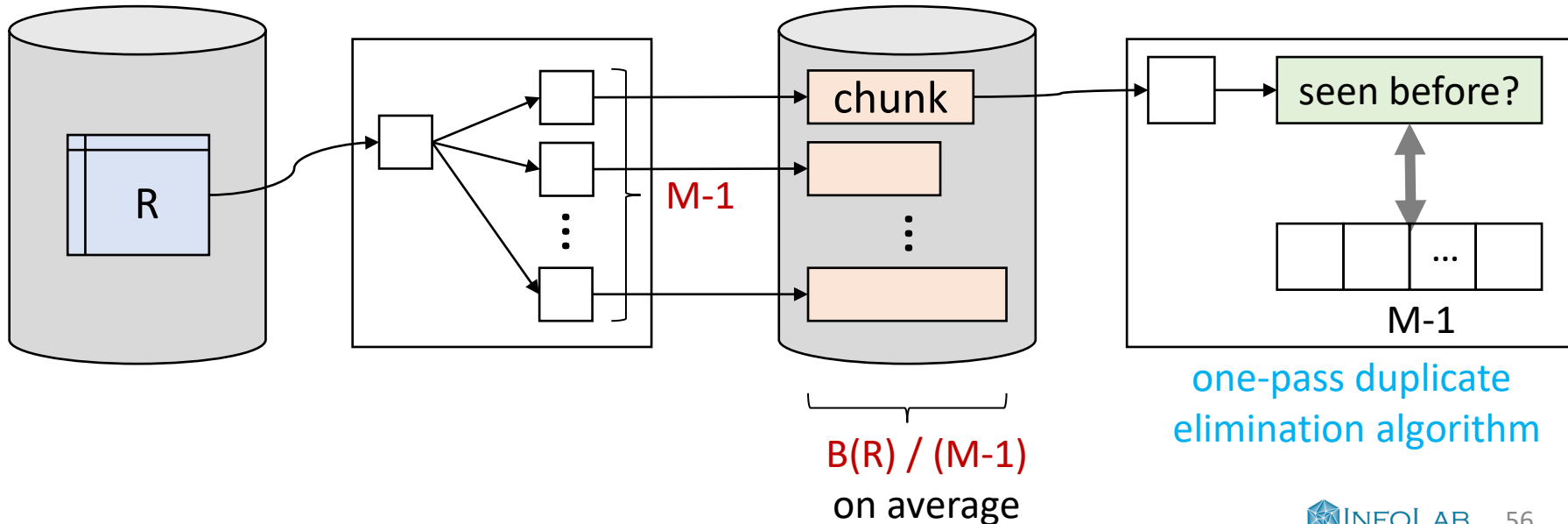
Hash-based duplicate elimination

- Pass 1: partition R to M-1 buckets (read and write, $2B(R)$)



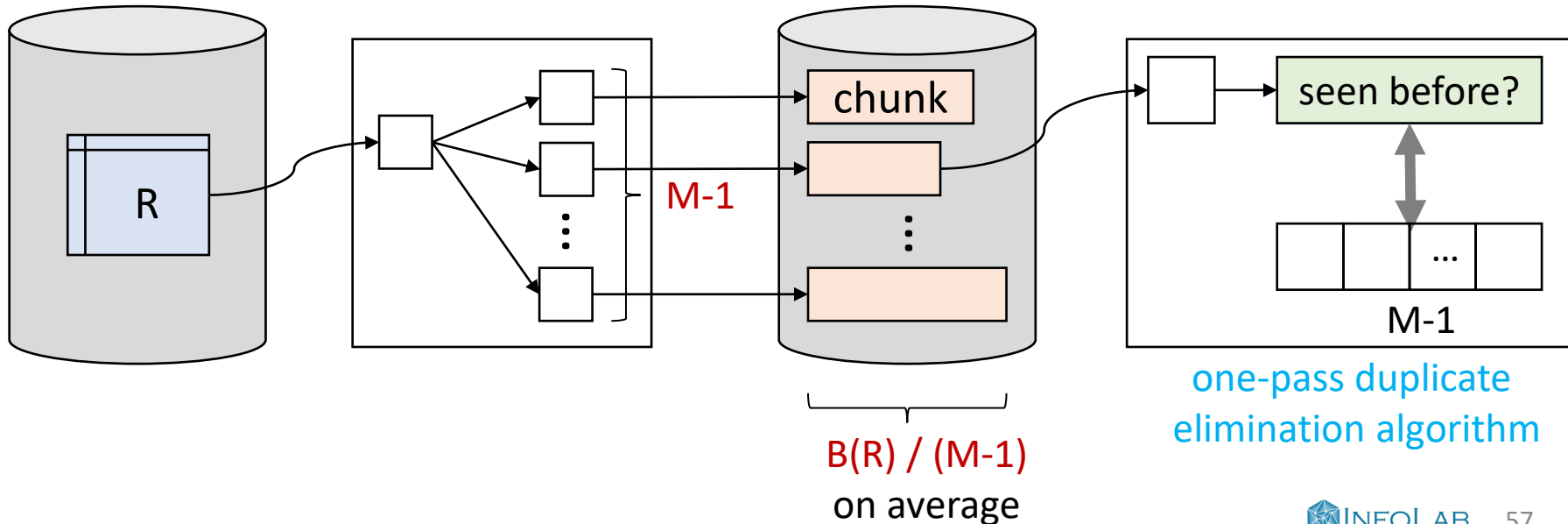
Hash-based duplicate elimination

- Pass 1: partition R to $M-1$ buckets (read and write, $2B(R)$)
- Pass 2: perform one-pass δ on each bucket (read, $1B(R)$)
 - $B(\delta(R_i)) \leq M$, for $0 \leq i < M$



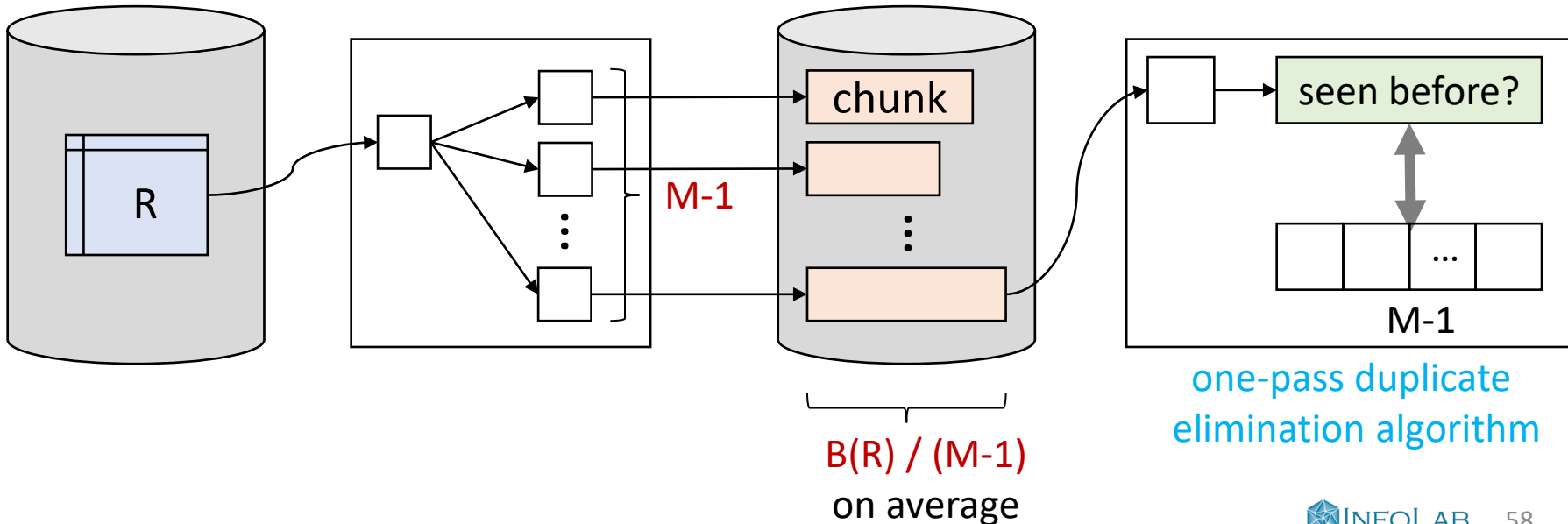
Hash-based duplicate elimination

- Pass 1: partition R to $M-1$ buckets (read and write, $2B(R)$)
- Pass 2: perform one-pass δ on each bucket (read, $1B(R)$)
 - $B(\delta(R_i)) \leq M$, for $0 \leq i < M$



Hash-based duplicate elimination

- Pass 1: partition R to $M-1$ buckets (read and write, $2B(R)$)
- Pass 2: perform one-pass δ on each bucket (read, $1B(R)$)
 - $B(\delta(R_i)) \leq M$, for $0 \leq i < M$
- Answer: $\bigcup_{0 \leq i < M} \delta(R_i)$

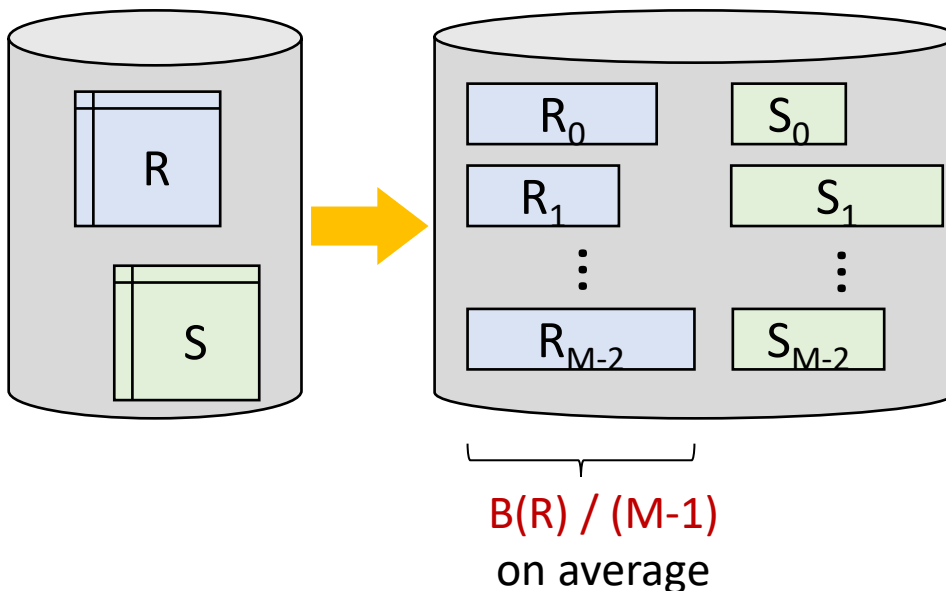


Hash-based grouping and aggregation

- Pass 1: partition R to $M-1$ buckets (read and write, $2B(R)$)
 - use the grouping attributes as hash key
- Pass 2: perform one-pass γ on each bucket (read, $1B(R)$)
 - actual limitation on the size of R is complex due to aggregation
 - usually can handle much larger R
 - conservative estimation: $B(R) \leq M^2$
 - # of disk I/O's : $3B(R)$

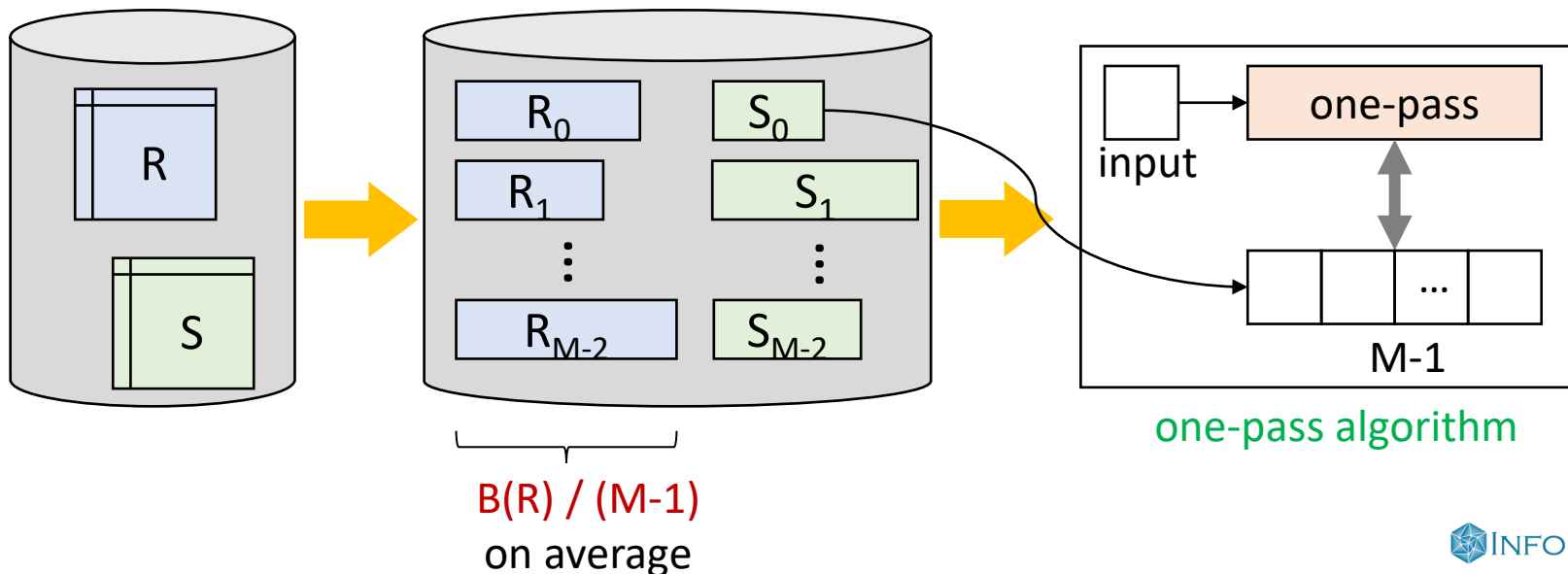
Hash-based union, intersection, and difference

- Pass 1: partition both R and S to $M-1$ buckets (the same $h()$)
- Pass 2: perform one-pass algorithm on each pair of buckets
- # of disk I/O's: $3(B(R)+B(S))$
- Memory limit: $\min(B(R_i), B(S_i)) \leq M$, for $0 \leq i < M$



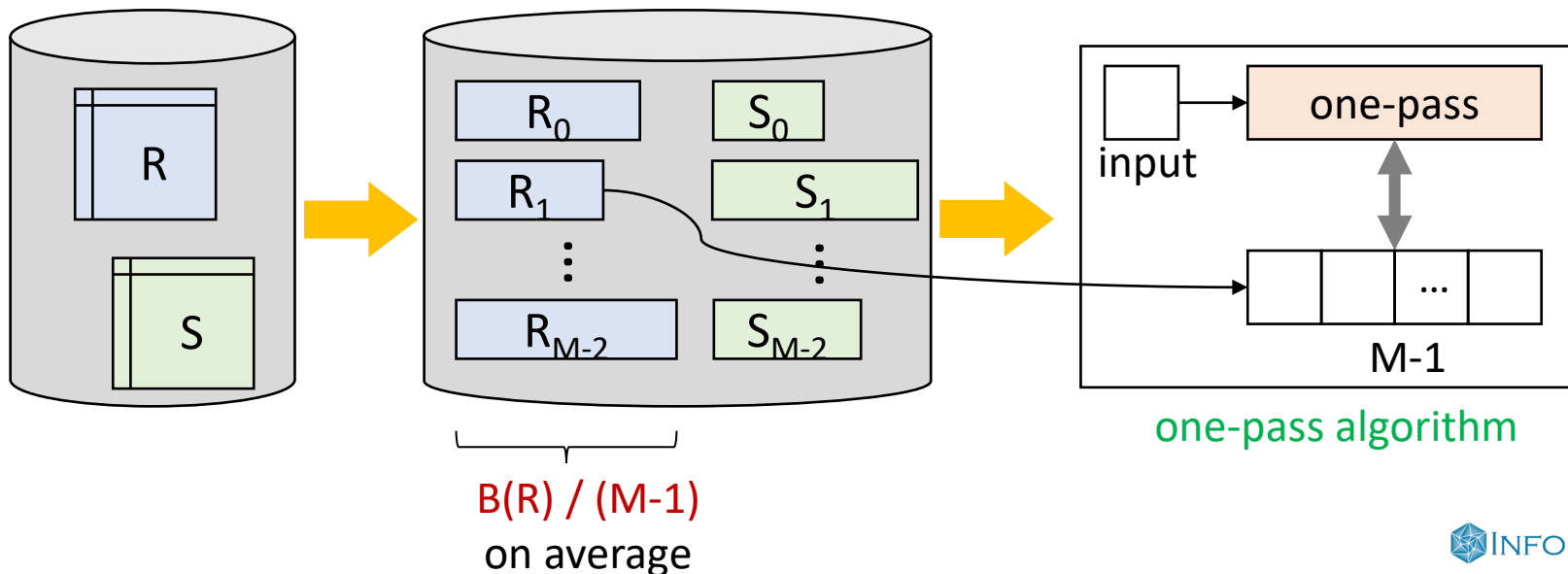
Hash-based union, intersection, and difference

- Pass 1: partition both R and S to M-1 buckets (the same $h()$)
- Pass 2: perform one-pass algorithm on each pair of buckets
- Disk I/O's: $3(B(R)+B(S))$
- Memory limit: $\min(B(R_i), B(S_i)) \leq M$, for $0 \leq i < M$



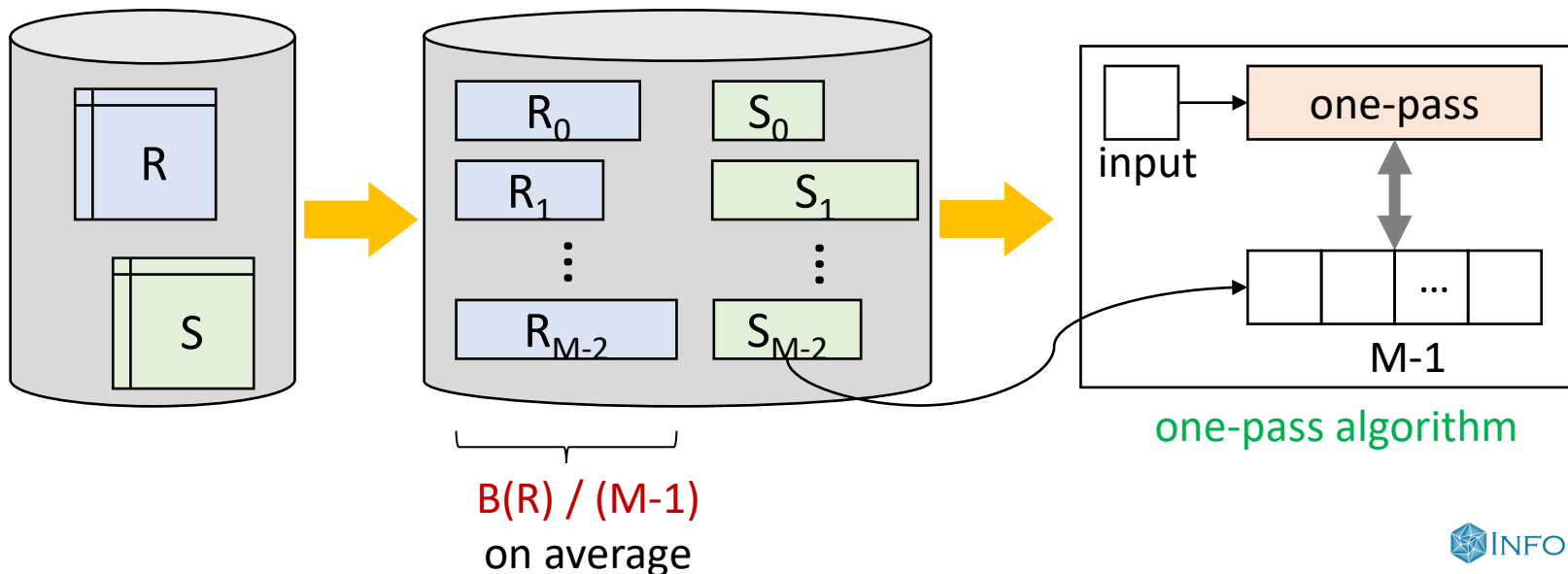
Hash-based union, intersection, and difference

- Pass 1: partition both R and S to M-1 buckets (the same $h()$)
- Pass 2: perform one-pass algorithm on each pair of buckets
- Disk I/O's: $3(B(R)+B(S))$
- Memory limit: $\min(B(R_i), B(S_i)) \leq M$, for $0 \leq i < M$



Hash-based union, intersection, and difference

- Pass 1: partition both R and S to M-1 buckets (the same $h()$)
- Pass 2: perform one-pass algorithm on each pair of buckets
- Disk I/O's: $3(B(R)+B(S))$
- Memory limit: $\min(B(R_i), B(S_i)) \leq M$, for $0 \leq i < M$



Hash-based join (**hash-join**)

- Pass 1: partition both R and S to M-1 buckets
 - hash key: join attribute
- Pass 2: perform hash-join on each pair of buckets
 - build a hash table for a smaller chunk
 - probe the tuples of a larger chunk against the hash table
- Disk I/O's: $3(B(R)+B(S))$
- Memory limit: $\min(B(R_i), B(S_i)) \leq M$, for $0 \leq i < M$
- Example: $B(R)=1000$, $B(S)=500$, $M=101$
 - # of I/O for bucketizing R: $2 \times 1000 = 2000$
 - # of I/O for bucketizing S: $2 \times 500 = 1000$
 - # of I/O for hash-join $\langle R_i, S_i \rangle$: $1000 + 500 = 1500$
 - total # of I/O: **4500** (= the cost of sort-merge-join in p.51)

Summary of hash-based algorithms

Operators	Total disk I/O's	Max sizes of R (and S) that we can perform the operation
γ, δ	$3B(R)$	$B(R_i) \leq M$
$\cup, \cap, -$	$3(B(R) + B(S))$	$\min(B(R_i), B(S_i)) \leq M$
$\bowtie$	$3(B(R) + B(S))$	$\min(B(R_i), B(S_i)) \leq M$

τ, γ, δ	$3B(R)$	$B(R) \leq M^2$
$\cup, \cap, -$	$3(B(R) + B(S))$	$B(R) + B(S) \leq M^2$
$\bowtie$ (improved)	$3(B(R) + B(S))$	$B(R) + B(S) \leq M^2$

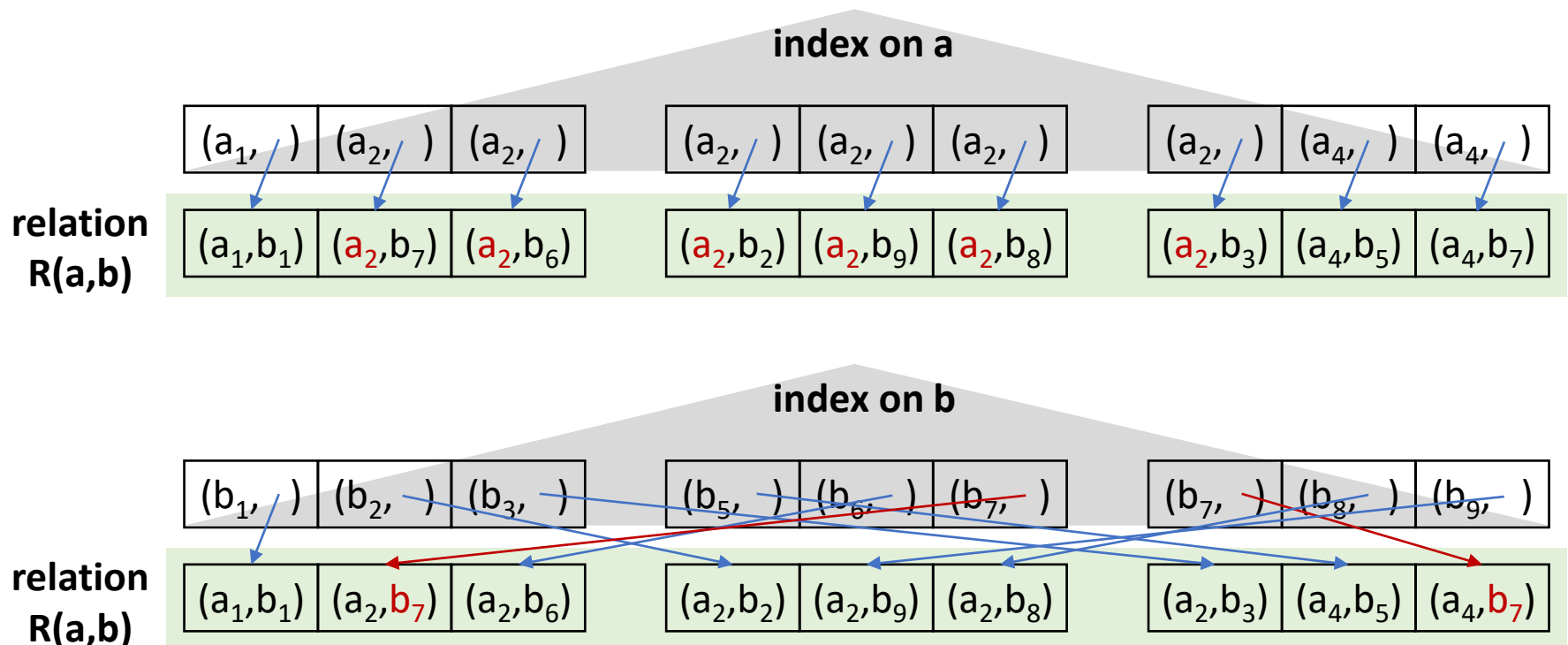
* sort-based algorithms

Two-Pass Algorithms

	iteration-based	sort-based	hash-based	index-based
one-pass	selection, projection, duplicate-elimination, grouping, union, intersection, difference, join, product			
two-pass	nested-loop joins	$\tau, \gamma, \delta,$ $\cup, \cap, -$, sort-based join	$\gamma, \delta,$ $\cup, \cap, -$, hash-based join	σ , index-based join

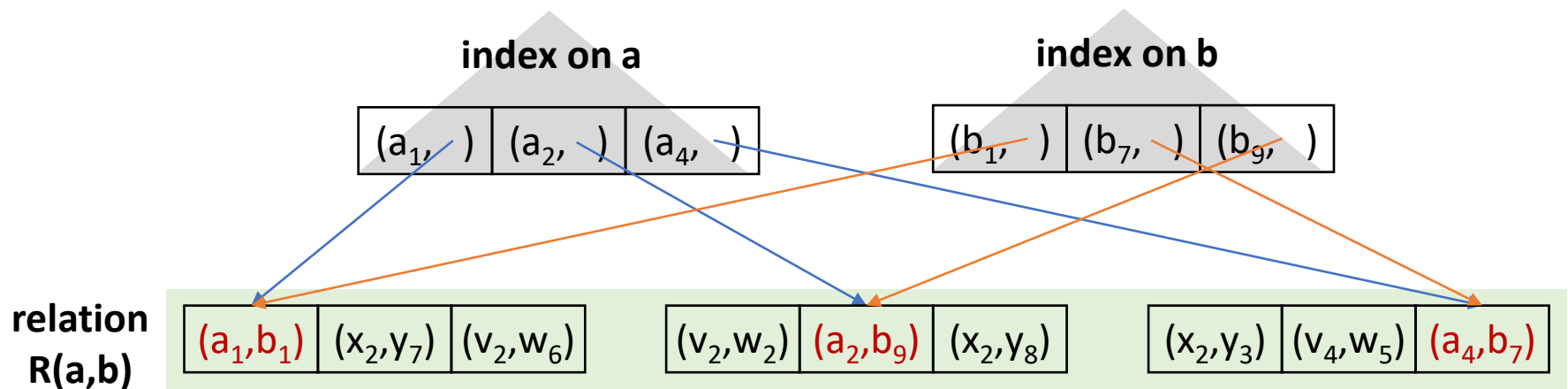
Clustering vs. nonclustering index (1)

- Clustering index: tuples appear on roughly as few blocks as can hold them, in terms of indexing attribute(s)
- A clustered relation may have a nonclustering index



Clustering vs. nonclustering index (2)

- Non-clustered relation CANNOT have an actual clustering index



Example of selection operation (σ)

- $V(R, a)$: # of unique values in the attribute a of R
- $T(R)$: # of tuples of R
- Example: $B(R)=1000$, $T(R)=20000$, 20 tuples/block, $\sigma_{a=0}(R)$

clustering of R	using index	clustering of index	$V(R, a)$	# of disk I/O's
yes	no			1000
no	no			20000
yes	yes	yes	100	10
don't care	yes	no	10	2000
don't care	yes	don't care	20000	1

we assume that the cost to access the index is zero

Index-based join (R has no index, not sorted)

- $R(X,Y) \bowtie S(Y,Z)$, index on the attribute $S.Y$

– for each $t \in R$, must read an average of $T(S) / V(S, Y)$ tuples of S

for reading S { index is nonclustering, $T(R) \cdot T(S) / V(S, Y)$
index is clustering, $\begin{cases} T(R) \cdot B(S) / V(S, Y), & \text{if } B(S) / V(S, Y) \geq 1 \\ T(R) & \text{if } B(S) / V(S, Y) < 1 \end{cases}$

- Example: $B(R)=1000$, $T(R)=10000$, $B(S)=500$, $T(S)=5000$, $V(S,Y)=100$, R is clustered, index on $S.Y$ is clustering

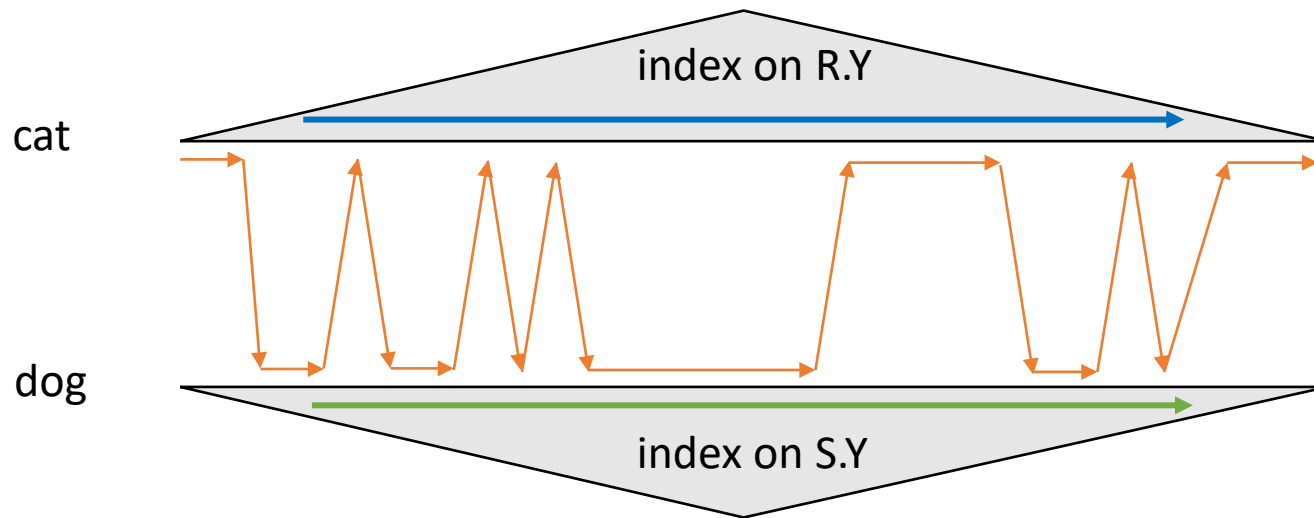
– # of I/O for reading R : 1000
– # of I/O for reading S : $10000 \times 500 / 100 = 50000$
– total # of I/O: 51,000 (> the cost 4,500 in p.51 and p.64)

Index-based join (with sorting R)

- Example: $B(R)=1000$, $T(R)=10000$, $B(S)=500$, $T(R)=5000$,
 $V(S,Y)=100$, B-tree index for S.Y is clustering
 - # of I/O for sorting R (phase 1): $2 \times 1000 = 2000$
 - create 10 sorted sublists
 - # of I/O for reading R: 1000 (explicitly sorted)
 - # of I/O for reading S: 500 (due to the clustering index)
 - total # of I/O: 3500 (< the cost 4,500 in p.51 and p.64)

Zig-zag join (both R and S have indexes)

- Example: $B(R)=1000$, $T(R)=10000$, $B(S)=500$, $T(S)=5000$,
indexes for both R.Y and S.Y are clustering
 - total # of I/O: 1500 (actually considerably less than 1500)



Thank you

Q & A